

Ejercicio 1. Control de Versiones con Git y GitHub

Omar Flores Olivares

16 de septiembre de 2026

Parte A. Investigación

1. Sistema de Control de Versiones (VCS)

Un **Sistema de Control de Versiones** (VCS, por sus siglas en inglés) es una herramienta de software que funciona como una línea del tiempo detallada, registrando el historial de modificaciones realizadas sobre un conjunto de archivos o código fuente a lo largo del tiempo.

En un entorno de trabajo colaborativo, el problema concreto que resuelve es la sobreescritura accidental de código y el caos generado por el manejo manual de versiones (por ejemplo, el uso de copias duplicadas). Permite que múltiples desarrolladores trabajen de manera simultánea e independiente en un mismo proyecto sin interferir entre sí.

De acuerdo con la documentación oficial de GitHub Docs, un sistema de control de versiones permite responder con precisión a las siguientes preguntas clave:

- ¿Qué cambios se realizaron?
- ¿Quién realizó los cambios?
- ¿Cuándo se aplicaron dichas modificaciones?
- ¿Por qué se llevaron a cabo los cambios?

2. Diferencia entre Git y GitHub

Aunque frecuentemente se utilizan como sinónimos, **Git** y **GitHub** son herramientas distintas con propósitos complementarios:

- **Git:** Es el sistema de control de versiones distribuido que se ejecuta de forma local en el equipo del desarrollador (usualmente mediante la terminal o línea de comandos). Git se encarga del rastreo de cambios, la creación de ramas, la gestión de *commits* y el mantenimiento de todo el historial del proyecto directamente en el almacenamiento local, funcionando al 100 % sin necesidad de una conexión a Internet.
- **GitHub:** Es un servicio en la nube y una plataforma web que aloja repositorios de Git remotos. Proporciona una interfaz gráfica para visualizar el código y el historial, y facilita la colaboración entre múltiples desarrolladores. Además, añade herramientas de gestión de proyectos que no forman parte de Git nativo, tales como la revisión de código mediante *Pull Requests*, el seguimiento de problemas (*issues*) y el control de accesos para los colaboradores.

En resumen, Git es el motor y la herramienta técnica que gestiona el historial localmente, mientras que GitHub es el entorno en la nube que permite centralizar y sincronizar esos repositorios locales para el trabajo en equipo.

3. Definiciones y Ejemplos

- **Repositorio:** Es la carpeta que contiene todos los archivos del proyecto, junto con el historial completo de cambios registrados por Git. Por ejemplo, la carpeta donde estoy guardando todas las tareas del curso de Bases de Datos.
- **Confirmación (*commit*):** Es una captura o "fotografía" permanente del estado del proyecto en un momento específico, acompañada de un mensaje explicativo. Por ejemplo:
 - Guardar los avances con el mensaje:
Definir estructura de la práctica 1 y hacer ejercicio 1 y 2.
- **Rama (*branch*):** Es una línea de desarrollo independiente y aislada del código principal, creada para probar cambios o agregar funciones sin afectar la versión funcional. Por ejemplo:
 - Crear la rama `actualizacion-readme` para modificar la presentación sin alterar la rama `main`.
- **Fusión (*merge*):** Es la operación que une el historial y los cambios de una rama secundaria dentro de otra rama (generalmente la principal). Por ejemplo:
 - Integrar los cambios de la rama `actualizacion-readme` de vuelta a la rama `main` tras concluir la edición.

- **Conflicto de fusión:** Ocurre cuando Git no puede combinar dos ramas automáticamente porque se modificó exactamente la misma línea de un archivo de formas distintas en ambas ramas. Por ejemplo:
 - Si dos integrantes editan simultáneamente el título del proyecto en la línea 1 del archivo `README.md` con textos diferentes.
- **Pull Request (PR):** Es una propuesta formal en GitHub donde un desarrollador solicita que los cambios de su rama sean revisados por el equipo antes de fusionarse. Por ejemplo:
 - Enviar una solicitud en GitHub para que tu compañero revise los ejercicios antes de unirlos a la rama principal.
- **Archivo .gitignore:** Es un archivo de configuración de texto plano que indica qué archivos o carpetas locales deben ser ignorados por Git y no subirse al repositorio. Por ejemplo:
 - Excluir archivos auxiliares de LaTeX (`*.aux`, `*.log`) para mantener limpio el repositorio.
- **Archivo README:** Es el documento Markdown principal de presentación del repositorio; explica la estructura, el propósito del proyecto e instrucciones de uso. Por ejemplo:
 - Un archivo `README.md` en la raíz de la práctica que detalla tu nombre, boleta y el índice de ejercicios.

4. Flujo de Trabajo Basado en Ramas y Revisión entre Pares

Un **flujo de trabajo basado en ramas** (*branch-based workflow*) es una metodología de desarrollo colaborativo que establece que la rama principal (`main` o `master`) debe contener exclusivamente código estable, probado y listo para producción. Bajo este esquema, queda estrictamente prohibido realizar cambios directos sobre la rama principal. En su lugar, cada nueva característica, corrección de errores (*bugfix*) o experimento debe desarrollarse en una rama secundaria independiente creada a partir de `main`.

Una vez que el trabajo en la rama secundaria se completa y se prueba localmente, se inicia el proceso de integración mediante la apertura de un *Pull Request* (PR). Es en este punto donde entra en juego la **revisión de código entre pares** (*peer review*).

La revisión entre pares antes de la fusión (*merge*) es una práctica fundamental en la ingeniería de software por las siguientes razones:

- **Aseguramiento de la calidad y detección temprana de errores:** Permite que otros desarrolladores identifiquen fallos lógicos, problemas de rendimiento o casos límite que el autor original pudo haber pasado por alto.

- **Estandarización y mantenibilidad:** Garantiza que el código cumpla con las convenciones de estilo, arquitectura y documentación establecidas por el equipo antes de formar parte de la base de código definitiva.
- **Transferencia de conocimiento:** Facilita que todos los miembros del equipo conozcan las distintas partes del sistema, evitando que el conocimiento sobre un módulo específico dependa de una sola persona.
- **Prevención de conflictos y regresiones:** Ayuda a evaluar el impacto que tendrán los nuevos cambios sobre las funcionalidades existentes antes de consolidarlos en la rama principal.

Fuentes Consultadas

- GitHub Docs. (2026). *GitHub Get Started*. <https://docs.github.com/es>